

FemboyEngine2D

Architecture Specification

Release
Version 0.3.0-alpha.1
2026-11-05
FBY-ARCH-0.1

Data-Oriented Design · Vulkan MDI · Compute LBS · EnTT · Box2D v3 · Rollback v3 · ImNodes · AOT

Document control

Document ID	FBY-ARCH-0.1
Version	0.3.0-alpha.1
Status	Alpha
Date	2026-11-05
Owner	Razraboychik
Audience	Engine, editor, tools and network engineers
Scope	Runtime, RHI, ECS, physics, network, tooling, visual graphs

Architecture Specification

This edition is generated from the same content model as the other language edition. Every run carries its language tag, so a translation tool or a proofing pass can process the document independently.

Contents

1. Executive summary
2. Repository hierarchy and build targets
3. Core definitions and the type system
4. RHI: Vulkan multi-draw and compute skinning
5. ECS, schedule and Snapshot v4
6. Physics: Box2D v3, DOD backend, chains, cloth and water
7. Network rollback and Snapshot v4
8. Asset pipeline and editor infrastructure
9. Visual graphs and the AOT transcriber
10. Quality gates and verification
11. Localization and documentation
12. Next steps after the Release
A. Invariants and capacities
B. Glossary

1. Executive summary

FemboyEngine2D is a C++26, data-oriented 2D engine at release-candidate status. The runtime is split into one static library per shipping role: a game links the engine, an editor links the engine plus the editor UI library, and the offline cooker links only the asset libraries. A shipped game never pulls in Dear ImGui, ImGuizmo, ImPlot or the TOML parser.

The design rules are deliberately strict. Simulation state is plain, trivially copyable data; hot paths use fixed capacity, function pointers and arena storage; frame paths do not allocate; and every subsystem exposes deterministic, checksummed state. Those rules are what make rollback, hot reload, snapshot testing and GPU multi-draw practical rather than aspirational.

- Core: Definitions.hpp, Vector2D/3D/4D, jobs, platform, plugin ABI, Mirror reflection.
- RHI: persistent frame rings, 48-byte sprite items, indirect mesh draws and one compute dispatch for linear-blend skinning.
- ECS and physics: EnTT registry, declarative SystemGraph, Snapshot v4, Box2D v3 bridge, joint chains, Verlet cloth and two water models.
- Network: NetRollbackSession with a 256-tick snapshot ring, input prediction, late-input replay and stable entity ids.
- Tooling: transactional CommandStack, AssetDatabase with .femeta rows, watcher, cooker, VFS and the ImNodes graph editors with an AOT transcriber.

2. Repository hierarchy and build targets

All source lives under Source/. The include root is Source itself, so every include names its domain explicitly. A header under Source/Engine/RHI is included as "Engine/RHI/RHI.hpp"; editor code uses "Editor/Panels/...". This removes the old ambient fby/ prefix and makes ownership visible in the first line of a file.

Path	Details
Source/Engine/Core	Definitions, value types, jobs, platform, plugins, Mirror
Source/Engine/RHI	Render hardware interface, shaders, materials, camera stack
Source/Engine/ECS	Registry, components, scenes, snapshots, skeleton, animation
Source/Engine/Input	Action mapper, contexts, gamepad snapshots, fixed input state
Source/Engine/Physics	Box2D v3 and DOD backends, joints, transport, wheels, cloth, fluid, particles
Source/Engine/Audio	Mixer, DSP graph, limiter, Ogg/Opus
Source/Engine/Network	NetRollbackSession, input rings, snapshot replication
Source/Engine/Assets	VFS, .femeta, cooker, localization, texture/audio formats
Source/Engine/Script	Bytecode VM, graph compiler, AOT backend
Source/Engine/UI	PinkUI, hypertext, font atlas, accessibility
Source/Engine/Video	WebM/VP9 playback and streaming

Path	Details
Source/Editor/Panels	Editor panels and ImNodes canvases
Source/Editor/Infrastructure	Command stack, cooker watcher, project management
Source/Apps, Source/Tests, Source/Tools, Source/Plugins	Applications, tests, offline tools, plugin fixtures

File names are PascalCase without exception: Vector2D.hpp, NetRollbackSession.cpp, ComponentRegistry.cpp. A style gate rejects a non-PascalCase stem, lowercase underscore names, non-Allman braces, trailing comments, Cyrillic text, dynamic_cast, typeid and any surviving Vec2, Vec3 or Vec4 identifier.

Target	Details
FemboyEngine2D::Engine	Runtime library; no editor UI, no TOML parser
FemboyEngine2D::Editor	Editor panels, command stack, ImGui/ImNodes integration
FemboyEngine2D::AssetCook	BC7/ASTC texture cooking shared by editor and offline tools
FemboyEngine2D::AssetLib	Pack and localization compilation; never shipped in a game
fby_shaderc_core	glslang wrapper reused in-process by the editor
fby_sandbox, fby_editor, fby_* tools	Applications, offline cookers, AOT compiler, scene tool

3. Core definitions and the type system

Source/Engine/Core/Definitions.hpp is the single source of truth for the low-level vocabulary. It defines FORCEINLINE, the static cast helper FemCast<>, the checked polymorphic cast Cast<>, the stable TypeId and the TypeObject base. It includes Mirror and the core value types, so a translation unit that includes it has the complete contract without a second include.

```
using TypeId = std::uint64_t;
template <typename T> consteval TypeId StableTypeId() noexcept;

template <typename To, typename From>
[[nodiscard]] FORCEINLINE To FemCast(From&& InValue) noexcept;
```

```
template <typename To, typename From>
[[nodiscard]] FORCEINLINE To* Cast(From* InObject) noexcept;
```

StableTypeId<T>() hashes Mirror's component name with FNV-1a. Reflected components therefore share one identity between the inspector, snapshot schema, plugin ABI and rollback checksum. A non-component class can pin a compiler-independent name with FBY_TYPE_NAME or accept the compiler signature hash. Cast<> upcasts at compile time and walks a flat parent table on a downcast; a wrong downcast returns nullptr and no compiler-generated type information is read. dynamic_cast and typeid are absent from the engine sources, and the style gate enforces that.

Core value types are trivially copyable and standard layout. The coordinate types are Vector2D, Vector3D and Vector4D, with uppercase X, Y, Z and W components. Rect is a half-open axis-aligned box; Color32 is one packed uint32 in 0xAABBGGRR order; Handle<Tag> is a nullable 32-bit pool index, never a pointer. Every component is checked by the compiler for trivial copyability and standard layout before it can enter a snapshot or a plugin table.

Layer	Details
Jobs	std::jthread pool, function-pointer tasks, ParallelFor with caller participation
Platform	SDL3 window/input, native dialogs, Vulkan surface creation
Plugins	C ABI, layout hash validation, registry supplied by the host
Mirror	consteval component descriptors from member pointers, no P2996 dependency

4. RHI: Vulkan multi-draw and compute skinning

The RHI accepts a FramePacket of plain data and never sees a component, an entity or a string. A sprite is one 48-byte RenderItem: position, size, uvMin, uvMax, layer, packed colour, rotation and flags. Items are sorted once per frame by a 64-bit key (16 bits layer, 40 bits depth, 8 bits sequence) and uploaded into a persistently mapped instance ring. The GPU reads one item per instance; there is no per-sprite vertex buffer, descriptor set or texture bind.

The mesh path is parallel and deliberately different. Static geometry is uploaded once into a device-local arena and addressed through MeshHandle. Deformable SkeletalAnimation/SkinComponent vertices are streamed into a host-visible ring; one compute dispatch runs linear-blend skinning for the whole frame; the result lands in a device-local SkinnedVertex buffer. MeshVertex is 40 bytes, SkinnedVertex is 32 bytes, and both sides use a full column-major Mat4 so a 2.5D camera can produce real z and perspective divide.

```
MeshVertex     40 B  position, uv, color, skin range, bone indices, weights
SkinnedVertex  32 B  homogeneous position (vec4), uv, color
RenderItem     48 B  position, size, uvMin, uvMax, layer, color, rotation, flags
```

Draws are emitted as one vkCmdDrawIndexedIndirect per (texture array, layer, geometry class). Per-draw model, layer and material data live in a storage buffer indexed by gl_InstanceIndex, and the

indirect command's firstInstance selects the record. That is why one multi-draw can carry a different model matrix per mesh without a per-mesh push constant. The Vulkan backend requires shaderDrawParameters; the null backend still runs the full frame packet path so tests exercise real data, not a mock.

Backend	Details
Vulkan	Windows/Linux/Android; indirect mesh path, compute LBS, runtime loader
Metal	macOS/iOS; same FramePacket contract, Metal argument buffers
Null	Headless CI and servers; full packet/ring path, no device

Three frames in flight are the default. Each slice owns an instance ring, a mesh vertex ring and a mesh index ring allocated once at device creation. Capacity is a startup decision, not a per-frame allocation: the defaults cover 1,048,576 sprite items, 65,536 mesh vertices and 131,072 mesh indices per slice.

5. ECS, schedule and Snapshot v4

The ECS is EnTT's registry with a strict component contract: trivially copyable, standard layout, no string, no container, no vtable. Components are data; behaviour lives in systems. The frame has four phases: PreUpdate (variable dt, input and streaming), FixedUpdate (constant dt, gameplay and physics), PostUpdate (camera, culling, audio) and Render (extract and editor).

SystemGraph is the production scheduler. Dependencies are explicit after/before bitmasks; a one-time topological sort produces the order; a cycle or a cross-phase dependency fails Build(). The graph holds up to 64 nodes and uses function pointers plus an opaque user pointer, never std::function. PluginHost removes every node whose user pointer matches when a plugin unloads.

Phase	Details
PreUpdate	Input, window events, asset streaming; variable dt
FixedUpdate	Gameplay, Box2D, joints, cloth, fluid; 0..N steps of constant dt
PostUpdate	Camera follow, culling, mixer submission; variable dt
Render	Render extract, UI build, editor draw; once per frame

Snapshot v4 is a canonical, checksummed byte image. The 104-byte header carries magic, version, entity/component counts, tick/baseTick, payload bytes, FNV-1a checksum, base checksum, schema hash, the RNG state and the v3 world-extension fields. ECS records come first, field-by-field through Mirror; the extension bytes for chains, cloth, water and emitter clocks follow and are addressed by

stable subsystem ids, never by Box2D handles. A delta snapshot stores only changed records and still checksums the base it was derived from.

```
struct SnapshotHeader {
    char magic[8];              // FBYSNP3\0
    uint32 version;             // kSnapshotVersion
    uint32 flags;               // full or delta
    uint32 entityCount;
    uint32 componentCount;
    uint64 tick, baseTick;
    uint64 payloadBytes, checksum, baseChecksum, schemaHash;
    RngState rng;
    uint32 extensionBytes, extensionOffset;
    uint64 extensionChecksum;
};
```

Every registered component contributes a name, a stable type id, a Mirror layout hash, a canonical size and four function pointers (has, capture, restore, remove). The layout hash rejects a same-size field change; the schema hash rejects a different component set. Restore does not allocate: the snapshot arena and entity tables are fixed at Initialize().

6. Physics: Box2D v3, DOD backend, chains, cloth and water

The physics header does not include box2d.h. RigidBody2D and Collider2D store packed b2BodyId/b2ShapeId values in uint64_t, so gameplay, scene IO and the network layer compile without the C API. PhysicsSystem owns the world; TransportGroups, VehicleWheels and the joint systems are explicitly Box2D-aware translation units that keep the C API behind the same packed handles. Units are explicit: Box2D works in metres, the renderer in pixels, and PhysicsDesc::pixelsPerMeter is the single conversion point.

Model	Details
Rigid bodies	Box2D v3 world, fixed step, contact events, teleport flag
Rope / chain / spring graph	A joint graph of capsule links: chain, rope and spring connections with break limits, ties and repair
Cloth	32x32 Verlet grid, Gauss-Seidel constraints, overlap query against Box2D shapes
Water	GerstnerField analytic waves plus ShallowWaterGrid on mdspan
DOD backend	SoA AABB solver with a sorted uniform grid; no joints, body CCD or rotational contact response
Transport	Path/motion carriers with contact support and conveyor surfaces; Box2D weld joints when available

Model	Details
Wheels	Box2D wheel-joint suspension, per-wheel motor torque and braking; no implicit powertrain

A rope is not a special body. It is a chain of small rigid links joined by connections; every connection carries its own material (rigid distance, spring distance, revolute with limits, spring revolute), its own break limits in newtons and newton metres, optional damage accumulation and its own repair state. A chain is therefore a builder over a fixed joint graph: sections splice chain and rope runs, ties connect two links across chains or two plain bodies, gameplay cuts enter the deterministic command log, and a broken connection leaves the link entities alive. The system hard-caps chains, links and connections, allocates the pool once and never allocates on the frame path. It owns the link entities it creates and never the anchor entities; Shutdown order matters because destroying the Box2D world invalidates every stored id, and Update must run directly after the physics step to consume Box2D joint events.

Cloth is a Verlet mass-spring grid projected with a fixed number of Gauss-Seidel iterations, then pushed out of the existing Box2D shapes. It is not a second collision world and does not duplicate level geometry. The CPU writes MeshVertex positions; the renderer streams them through the same deformable path as a skinned mesh, so cloth adds no second draw path. A pinned vertex is simply invMass = 0.

Two water models share one system. GerstnerField is a small sum of trochoidal waves: deterministic, analytic, allocation-free, with a Jacobian that fades foam in at crests. ShallowWaterGrid stores a height/velocity field in mdspan views over fixed arrays and integrates the linearised shallow-water equations with a clamped CFL number. Neither model knows about the RHI; a renderer samples the field or uploads its height view as an R32F texture.

The DOD backend is a second implementation of the same PhysicsSystem contract, selected by project policy or PhysicsDesc.backend. It owns SoA positions, velocities and collider extents and resolves pairs through a sorted uniform grid; PhysicsSystem::Capabilities() is the only supported feature query. Transport and wheels explicitly declare their backend boundary: transport support and conveyors work on both solvers, while weld joints and wheel joints require Box2D.

7. Network rollback and Snapshot v4

NetRollbackSession is the glue between Snapshot v4, CommandLog, Pcg32 and the gameplay components (tags, attributes, effects and skills). A session owns a fixed ring of full snapshots (one per simulated tick), one input ring per player, a stable NetEntityId to local EnTT id map and the replay loop. Everything after Initialize() is allocation-free: the snapshot arena, entity tables, input rings and frame metadata are fixed capacity. The simulation callback is a function pointer plus void*, never std::function.

When a remote input arrives late, the session restores the snapshot immediately before the earliest corrected tick, restores the RNG and CommandLog cursor from the same slot, reapplies structural commands, resimulates the affected ticks with the corrected input and prediction for still-missing inputs, captures new snapshots, and reports which frames actually changed.

Constant	Details
kMaxNetPlayers = 8	Input ring and tick input array capacity
kMaxRollbackTicks = 256	Snapshot history depth for correction
kMaxNetEntities = 4096	Stable network entity table
kInputRingCapacity = 256	Late input overwrite window

NetInputFrame is 24 bytes and uses signed 16-bit fixed-point axes, not floats: a float bit pattern is not guaranteed to survive a different compiler, and the simulation must be a pure function of the input bits. The status of each player frame is Confirmed, Predicted or Missing. A late input overwrites the prediction in place at the same tick, which is exactly what replay needs.

NetEntityId is assigned by the authority (or by the local peer for locally owned objects) and is never derived from an EnTT id. A restored snapshot, a different spawn order or a different allocator must not change it. The replicated gameplay state is a compact tag mask plus fixed attribute, effect and skill records; the checksum folds each of them per mapped entity. CommandLog records structural spawn/despawn/add/remove, so a rollback can replay structure as well as values.

8. Asset pipeline and editor infrastructure

A component never stores a path. It stores a small, generation-checked AssetHandle that names one row of the AssetDatabase metadata table. The row owns the stable AssetId/GUID, resource kind, source and cooked paths, dependency list and import settings. AssetHandle::generation detects a reused slot; AssetMeta::contentGeneration lets a renderer or cache compare content versions. Hot reload never exposes a half-updated asset: edits are staged and committed atomically.

```
[[asset]]
guid = "018f0c2e-..."
type = "texture"
source = "atlas/orb.png"
cooked = "atlas/orb.ftex"
format = "bc7_srgb"
depends = ["atlas/atlas.toml"]

[import]
generate_mips = true
max_size = 2048
```

AssetDatabaseWatchSystem watches the source folders through the OS change API and pushes changed paths into a fixed SPSC ring. The watcher thread never mutates the database or the RHI; Poll() runs on the editor thread, resolves the AssetHandle, schedules a cook, stages the new metadata, commits it, bumps the content generation and issues reload notices. A loose PNG/WAV/WebM without a .femeta row is imported automatically with deterministic ids and default settings.

CommandStack is the editor's undo/redo engine. An inspector edit or an ImGuizmo drag is one transaction: BeginTransaction, Record one or more entities, Commit. A recorded component is a sparse byte delta, so moving one Vector2D in a 20 KB SkeletonComponent costs tens of bytes. Storage is a fixed command array plus one payload arena; no operator new, no std::function, no exceptions on the edit path. The stack is editor-only and never links into a game.

Tool	Details
VFS	Mounts .fempak archives and loose directories behind one path space
VFS	Deterministic read order, range reads, no per-file allocation on the hot path
fby_fempak	Texture/audio/localization cooker; byte-identical with the editor importer
fby_loc_tool	Localization TOML -> FlatBuffers pack
fby_scene_tool	Scene/prefab compiler and round-trip verifier
fby_aotc	Bytecode -> C++26 -> native plugin at cook time

9. Visual graphs and the AOT transcriber

The Material Graph is a small DAG stored in three flat fixed-capacity arrays: nodes, pins and params. A whole material is one trivially copyable value that can be memcpy'd, snapshotted by the undo system and hashed without an allocator. The compiler validates exactly one output node, connected required pins, type compatibility and acyclicity, then emits one GLSL temporary per node. No interpreter and no per-frame dispatch: a material costs exactly what its generated shader costs. The hash excludes UI positions, so dragging a node does not recompile a shader.

Limit	Details
MaterialGraph nodes = 128	A larger material is a design smell, not a reason to grow the runtime
MaterialGraph pins = 384	Every input slot of every node, contiguous per node
MaterialGraph params = 64	Scalar, vec2, vec3, colour, atlas layer and texture sample payloads
LogicGraph nodes = 128	Editor document is one 10 KiB POD
LogicGraph links = 256	Sparse link table; one transaction per edge edit

NodeCanvasPanel is a transactional ImNodes canvas. A panel supplies a CanvasModelOps table of plain function pointers and void*; the canvas never sees a virtual interface. The canvas owns only interaction state and opens exactly one CommandStack transaction per drag or edge edit. Because the whole graph is registered as one external buffer, moving a node records an 8-byte Vector2D, not the whole 7 KiB graph.

VisualScript compiles the LogicGraph to canonical bytecode with a Kahn topological pass; cycles are a compile error, never a runtime stack overflow. The VM executes the exact artifact the cooker ships. For release, AotTranspiler emits one flat C++26 translation unit with one function per event and zero engine dependencies beyond AotAbi.hpp; AotBuilder runs clang/g++ -O3 -shared on a background jthread and the editor polls its state without blocking the frame. The generated module carries the graph hash and every component LayoutHash; the loader validates them before the first call.

```
LogicGraph -> GraphCompiler -> bytecode -> VM (PIE)
                         \-> AotTranspiler -> C++26 -> clang/g++ -O3 -shared ->
plugin

MaterialGraph -> type-check -> GLSL -> glslang -> SPIR-V -> VkPipeline
```

10. Quality gates and verification

The Release is verified by the CTest suite: rollback replay, RNG/noise, network rollback, indirect mesh smoke, script VM, script AOT, PinkUI, DSP mixer, audio cook, material graph, video, texture cook, asset metadata, animation timeline, editor command stack, editor infrastructure, graph editor, physics extensions, UI accessibility, P3 integration and the Definitions contract. Tests are plain executables with no framework, so they run headless and on a console devkit.

Gate	Details
ctest	full suite passes; no GPU required for the headless subset
CheckStyle.py	PascalCase stems, required tree, no Vec2/3/4, no RTTI tokens, no Cyrillic, Allman, braces
clang-format	Allman, 4 spaces, 120 columns, mandatory braces
clang-tidy	Parameters In*, outputs Out*, locals Scoped*, no type shadowing
RTTI scan	dynamic_cast/typeid are rejected in source; no engine code uses compiler type information

The style gate intentionally checks the repository, not only a diff. A new file with a lowercase name, a Vec2 alias, a trailing comment or a Cyrillic comment fails the build review before it can reach main. The comments are English-only, one line, above the code and without a trailing period; the only remaining trailing comments are two NOLINT suppressions required by clang-tidy.

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo
-DFBY_BUILD_TESTS=ON
cmake --build build --parallel 8
```

```
ctest --test-dir build --output-on-failure
python Source/Tools/Dev/CheckStyle.py
```

11. Localization and documentation

This document is generated by Documentation/Tools/GenerateDocumentation.py. The script owns one language-neutral content model and renders it to independent English and Russian editions in Markdown, Word and PDF. Each edition has its own DOCX and PDF; the PDF is exported from that language's DOCX by LibreOffice Kit.

Styles are language-neutral: FBY Title, FBY Subtitle, FBY Heading 1-3, FBY Body, FBY Code, FBY Caption, FBY Table Header, FBY Table Body and FBY Note. Every run carries an explicit w:lang tag (en-US or ru-RU), and the body/heading styles set ascii, hAnsi, eastAsia and cs fonts. Adding a language means adding a translation column and a new section; it does not require a style rewrite or a per-language layout.

Artifact	Details
FemboyEngineArchitecture.md	Combined document with both language parts
FemboyEngineArchitecture_EN.md	Standalone English Markdown
FemboyEngineArchitecture_RU.md	Standalone Russian Markdown
FemboyEngineArchitecture.docx	Word document with two independent sections
FemboyEngineArchitecture.pdf	PDF exported from the Word document

```
python Documentation/Tools/GenerateDocumentation.py
python Documentation/Tools/GenerateDocumentation.py --skip-pdf
python Documentation/Tools/GenerateDocumentation.py --node <node> --lo-cli
<libreoffice-kit cli.js>
```

12. Next steps after the Release

- Promote CheckStyle.py to a CI test and a pre-commit hook, so a lowercase file or a Vec2 alias cannot merge.
- Add a public C++ API stability policy: frozen component layouts, Snapshot v4 compatibility tests and an ABI-version matrix for plugins.
- Finish Metal parity: indirect mesh path, compute skinning and the same FramePacket contract as Vulkan, then enable macOS/iOS CI.
- Add snapshot delta compression and a bandwidth budget, with a replay viewer that steps the snapshot ring and overlays corrected frames.

- Move cloth and shallow water to an optional GPU solver, keeping the CPU Verlet path as the deterministic fallback used by rollback.
- Add a dependency-graph view to the asset database, with cycle detection and cook-order export.
- Add editor localization: the same language-tagged style system for panel strings, with hot reload from a .fbin catalogue.
- Add a deterministic replay/fuzz harness to the CI matrix and a performance regression budget for the frame path.

A. Invariants and capacities

Invariant	Details
RenderItem	48 bytes, trivially copyable, fixed GPU layout
MeshVertex / SkinnedVertex	40 / 32 bytes, std430-compatible
SnapshotHeader	104 bytes, little-endian, FNV-1a checksum
SystemGraph nodes	64, compile-time capacity, one-time topological sort
Net players / ticks / entities	8 / 256 / 4096, fixed rings and tables
MaterialGraph nodes / pins / params	128 / 384 / 64, flat arrays
LogicGraph nodes / links	128 / 256, one 10 KiB POD document
CommandStack commands / payload	512 commands, 4 MiB default payload arena

B. Glossary

Term	Details
DOD	Data-oriented design: data is flat, hot paths are allocation-free and cache-friendly
MDI	Multi-draw indirect: one indexed indirect command carries many per-instance draws
LBS	Linear-blend skinning: palette[bone] * bindPosition in one compute pass

Term	Details
.femeta	Per-asset metadata row: stable id, source/cooked paths, dependencies, import settings
AOT	Ahead-of-time: bytecode translated to a native shared library at cook time
PIE	Play-in-editor: the editor runs the game simulation in its own viewport